TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Program Using Recursive Function — Includes Parameter Passing to Flow Chart

A function that calls itself: the base case that stops it, the call stack that remembers each unfinished caller — and the full story of parameter passing, from call by value to the pointer that finally lets a function change your variable. The recursive algorithm is drawn as a flowchart, where the function call appears as the predefined-process symbol.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 9 · LEVEL 2 TERM 1

1. Aim

To write recursive functions (factorial, Fibonacci), trace them through the call stack, draw the recursive algorithm as a flowchart, and demonstrate both parameter-passing modes — call by value and call by reference — in one program.

2. Concepts you need

- **Recursion:** a function that calls itself on a *smaller* version of the same problem. Two ingredients are compulsory: a **base case** that answers directly without calling again, and a **recursive case** that moves toward the base case. Miss either and the calls never stop.
- **The call stack:** every call gets its own fresh copies of the parameters and local variables, stacked while it waits for the call it made. The winding phase piles calls up; the unwinding phase hands answers back, multiplying as it goes. factorial(4) is four stacked calls, not one call "looping".
- **Parameter passing — value:** the default. The argument's value is copied into the formal parameter; the function cannot change the caller's variable (Experiment 8, Task 3).
- **Parameter passing — reference (by pointer):** pass the variable's **address** with &a; the formal parameter is a pointer (int *x); inside, *x reads and writes the caller's own variable. This is how a C function "returns" more than one result — scanf has been doing it to you since Experiment 4.
- **In a flowchart**, a call to a named routine is drawn with the **predefined-process** symbol: a rectangle with a vertical line near each end. The recursive call is simply that symbol bearing the function's own name.

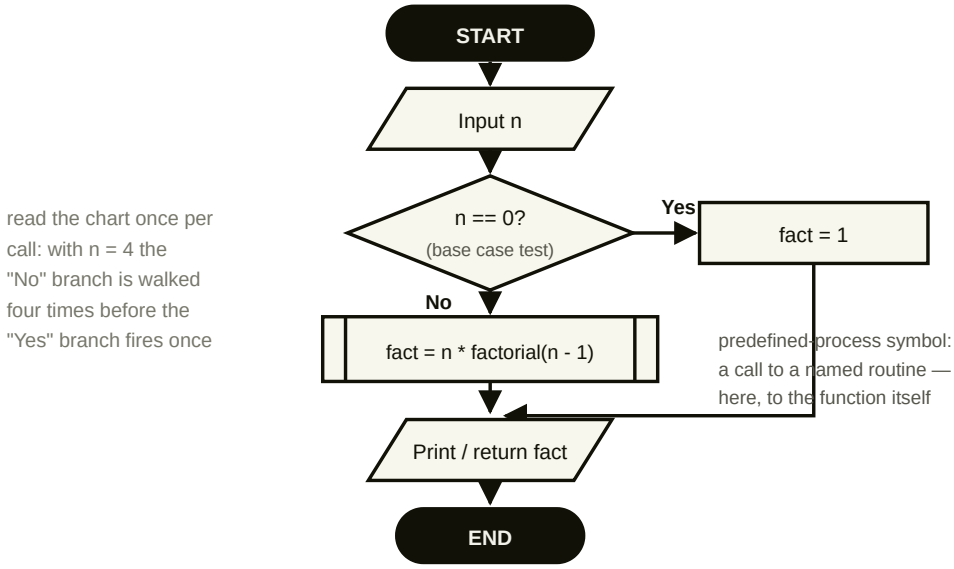


Figure 1 — Flowchart of recursive factorial. The chart describes ONE call; recursion means this same chart is re-entered with n reduced, stacking the unfinished multiplications until the base case fires.

3. Lab procedure

1. Predict each output; for the recursive programs, write the full winding/unwinding trace before running.
2. Add a printf inside factorial showing n at entry (in a copy); run for n = 4 and match the printed cascade against your trace and Figure 1.
3. Call factorial with n = −1 in the copy, observe the failure, and identify its cause precisely (be ready to state it in the viva).
4. For the swap program, run once; then delete the * characters inside swapRef in a copy, recompile, and explain any change in behaviour.

4. Worked examples

A — Recursive factorial, fully traced

```
#include <stdio.h>

long factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```

```
int main(void)
{
    printf("Factorial of 4 = %ld\n", factorial(4));
    return 0;
}
```

Output:

```
Factorial of 4 = 24
```

PHASE	WHAT HAPPENS
Winding	factorial(4) needs factorial(3); factorial(3) needs factorial(2); factorial(2) needs factorial(1); factorial(1) needs factorial(0). Four calls wait on the stack, each holding its own n.
Base case	factorial(0) returns 1 immediately — the only call that answers without asking.
Unwinding	factorial(1) = 1 × 1 = 1; factorial(2) = 2 × 1 = 2; factorial(3) = 3 × 2 = 6; factorial(4) = 4 × 6 = **24**.

B — Fibonacci by recursion

```
#include <stdio.h>

int fib(int n)
{
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    return fib(n - 1) + fib(n - 2);
}

int main(void)
{
    printf("fib(6) = %d\n", fib(6));
    return 0;
}
```

Output:

```
fib(6) = 8
```

Check against the sequence built by hand: fib(0)=0, 1, 1, 2, 3, 5, **8** — each term the sum of the previous two ✓. Note the cost: fib(6) makes two calls, each making two more; the repeated sub-calls (fib(4) is computed twice over) are why naive recursion is elegant but not always efficient.

C — Parameter passing: the swap that fails, and the swap that works

```c
#include <stdio.h>

void swapVal(int x, int y)
{
    int t = x;
    x = y;
    y = t;
}

void swapRef(int *x, int *y)
{
    int t = *x;
    *x = *y;
    *y = t;
}

int main(void)
{
    int a = 10, b = 20;
    swapVal(a, b);
    printf("After swapVal: a = %d, b = %d\n", a, b);
    swapRef(&a, &b);
    printf("After swapRef: a = %d, b = %d\n", a, b);
    return 0;
}
```

Output:

```
After swapVal: a = 10, b = 20
After swapRef: a = 20, b = 10
```

swapVal flawlessly swaps its own copies, which then vanish — main's a and b never moved. swapRef receives the *addresses* of a and b; *x and *y are a and b themselves under other names, so the swap is real. One program, both passing modes, opposite results — keep it as your reference example.

5. Practice tasks

1. Write sumTo(int n) recursively (sum of 1..n) and give the output for n = 10 with the unwinding trace.
2. Write gcd(int a, int b) recursively using Euclid's rule — gcd(a, b) = gcd(b, a mod b), base case b == 0 — and give the output and call trace for gcd(48, 36).
3. The factorial of Example A is called with n = −1. Explain exactly why it never returns, name the failure, and state the guard a careful programmer adds.

Answer key

1. ```c
   #include <stdio.h>

   int sumTo(int n)
   {
       if (n == 0)
           return 0;
       return n + sumTo(n - 1);
   }

   int main(void)
   {
       printf("Sum 1..10 = %d\n", sumTo(10));
       return 0;
   }
   ```

   Unwinding: sumTo(0) = 0; then 1+0 = 1, 2+1 = 3, 3+3 = 6, 4+6 = 10, 5+10 = 15, 6+15 = 21, 7+21 = 28, 8+28 = 36, 9+36 = 45, 10+45 = **55**. Output: **Sum 1..10 = 55** (agrees with 10×11/2 = 55).

2. ```c
   #include <stdio.h>

   int gcd(int a, int b)
   {
       if (b == 0)
           return a;
       return gcd(b, a % b);
   }

   int main(void)
   {
       printf("gcd(48, 36) = %d\n", gcd(48, 36));
       return 0;
   }
   ```

   Trace: gcd(48, 36) → gcd(36, 48%36 = 12) → gcd(12, 36%12 = 0) → base case returns 12. Output: **gcd(48, 36) = 12**.

3. With n = −1 the recursive case runs: factorial(−1) calls factorial(−2), then −3, … — n decreases away from the base case 0 and never equals it. Calls pile onto the stack until memory is exhausted: a **stack overflow**, and the program crashes without printing anything. The guard: validate the input before (or at entry to) the recursion — reject negative n (e.g. in main, or return an error for n < 0). A base case only protects you if every legal path reaches it.
